Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

РАЗРАБОТКА ПРОГРАММНОГО МОДУЛЯ ПОДДЕРЖКИ YDB В СИСТЕМЕ KEYCLOAK В КАЧЕСТВЕ ОСНОВНОГО ХРАНИЛИЩА ДАННЫХ

Автор: Мухтаров Айнур Дамирович _____

Направление подготовки: 01.03.02 Прикладная математика и информатика

Квалификация: Бакалавр

Руководитель ВКР: Аксенов В.Е., д.т.н. _____

Санкт-Петербург, 2026 г.

Обучающийся Мухтаров Айнур Дамирович
Группа М3437 Факультет ИТиП

Направленность (профиль), специализация
Информатика и программирование

ВКР принята «____» _____ 20___ г.

Оригинальность ВКР _____%

ВКР выполнена с оценкой _____

Дата защиты «» г.

Секретарь ГЭК Штумпф С. А. _____

Листов хранения _____

Демонстрационных материалов/Чертежей хранения _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
1. Анализ предметной области 7
1.1. Система управления идентификацией Keycloak 7
1.1.1. Архитектура Keycloak 7
1.1.2. Регистрация SPI 7
1.1.3. Ключевые SPI и ProviderFactory для работы с базой данных 10
1.1.4. Механизм миграций и DBLockProvider. 10
1.1.5. Модель данных Keycloak 11
1.2. СУБД YDB 11
1.2.1. Ключевые свойства YDB 11
1.2.2. Диалект Hibernate для YDB. 12
1.3. Аналоги и существующие решения 13
2. Проектирование решения. 15
2.1. Общая архитектура модуля 15
2.2. Принципы проектирования схемы данных 15
2.2.1. Анализ подходов к миграции схемы 15
2.3. Разработка диалекта Hibernate для YDB 17
2.4. Архитектура механизма повторного выполнения транзакций. 17
3. Реализация 18
3.1. Диалект Hibernate v7 для YDB 18
3.2. Реализация провайдеров Keycloak. 18
3.3. Управление схемой данных через Liquibase 18
3.4. Переопределение именованных запросов. 18
3.5. Тестирование. 18
ЗАКЛЮЧЕНИЕ 19

ВВЕДЕНИЕ

Системы управления идентификацией и доступом (Identity and Access Management, IAM) являются критически важным компонентом современных распределённых приложений. Среди таких систем особое место занимает Keycloak — решение с открытым исходным кодом, предоставляющее механизмы аутентификации, авторизации, управления пользователями и единого входа (SSO). Keycloak широко применяется в корпоративных системах, микросервисных архитектурах и облачных платформах.

По умолчанию Keycloak поддерживает реляционные СУБД: PostgreSQL, MySQL, MariaDB, Oracle, TiDB и Microsoft SQL Server. Хранение данных в реляционных СУБД в условиях высоких нагрузок и масштабирования может становиться узким местом системы: традиционные РСУБД ограничены в горизонтальном масштабировании и требуют сложных решений для обеспечения отказоустойчивости.

YDB (Yandex Database) — это распределённая СУБД, разработанная компанией Яндекс и выпущенная как программное обеспечение с открытым исходным кодом. YDB обеспечивает горизонтальное масштабирование, высокую доступность и строгую транзакционную семантику при распределённом выполнении. Архитектура YDB изначально ориентирована на работу в условиях высоких нагрузок, что делает её перспективной заменой традиционным РСУБД в сценариях с требованиями к масштабируемости.

Однако YDB имеет ряд архитектурных особенностей, отличающих её от стандартных реляционных СУБД:

— отсутствие поддержки внешних ключей на уровне базы данных;
— обязательное наличие явного первичного ключа в каждой таблице;
— специфический SQL-диалект YQL;
— отсутствие поддержки пессимистичных блокировок (`FOR UPDATE`);
— применение оптимистичного управления конкурентным доступом — конфликты транзакций обнаруживаются в момент коммита.

Целью работы является разработка программного модуля, обеспечивающего интеграцию Keycloak с YDB в качестве основного хранилища данных с сохранением функциональной совместимости и корректности работы системы.

Практическая значимость работы состоит в том, что разработанное расширение позволяет использовать распределённую СУБД YDB в качестве хранилища данных для Keycloak, что открывает возможности для горизонтального масштабирования IAM-компонента в крупных системах.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Система управления идентификацией Keycloak

Keycloak — это система управления идентификацией и доступом с открытым исходным кодом, разработанная компанией Red Hat. Система реализует протоколы OAuth 2.0, OpenID Connect и SAML 2.0, и предоставляет единую точку входа для множества приложений.

1.1.1. Архитектура Keycloak

Keycloak построен на основе Quarkus и использует Hibernate в качестве ORM-фреймворка для работы с базой данных. Управление схемой базы данных осуществляется через Liquibase.

Центральным элементом расширяемости Keycloak является механизм `Service Provider Interface (SPI)`. SPI позволяет заменять внутренние реализации компонентов пользовательскими, сохраняя совместимость с остальными частями системы.

1.1.2. Регистрация SPI

Для подключения собственной реализации через механизм SPI необходимо выполнить следующие шаги.

Шаг 1. Создать класс SPI, реализующий интерфейс `org.keycloak.provider.Spi`. Класс описывает точку расширения — его имя, тип провайдера и тип фабрики.

Например, SPI для подключения к базе данных через JPA выглядит следующим образом:

```
public class JpaConnectionSpi implements Spi {

    @Override
    public boolean isInternal() {
        return true;
    }

    @Override
    public String getName() {
        return "connectionsJpa";
    }

    @Override
```

```
    public Class<? extends Provider> getProviderClass() {
        return JpaConnectionProvider.class;
    }

    @Override
    public Class<? extends ProviderFactory>
       getProviderFactoryClass() {
        return JpaConnectionProviderFactory.class;
    }
}
```

Шаг 2. Зарегистрировать SPI и его реализацию через стандартный механизм Java `ServiceLoader`. В ресурсы приложения создаются файлы в директории `META-INF/services/`, содержащие полные имена классов реализаций:

```
META-INF/services/org.keycloak.provider.Spi
org.keycloak.connections.jpa.JpaConnectionSpi
```

Шаг 3. Реализовать интерфейс `ProviderFactory`. Фабрика управляет жизненным циклом провайдера и обязана реализовать следующие методы:

```
public interface ProviderFactory<T extends Provider> {
    T create(KeycloakSession session);
    void init(Config.Scope config);
    void postInit(KeycloakSessionFactory factory);
    void close();
    String getId();
    default int order() { return 0; }
}
```

а) `create` — создаёт экземпляр провайдера для каждой сессии;

б) `init` — вызывается один раз при старте сервера;

в) `postInit` — вызывается после инициализации всех фабрик;

г) `close` — вызывается при остановке сервера;

д) `getId` — возвращает уникальный идентификатор реализации;

е) `order` — приоритет среди фабрик (по умолчанию 0).

Шаг 3. Реализовать интерфейс `Provider`.

Интерфейс `Provider` — базовый интерфейс любого провайдера в Keycloak.

```
public interface Provider {
```

```
    void close();
}
```

Метод `close()` вызывается по завершении `KeycloakSession` — то есть по окончании каждого HTTP-запроса или транзакции. В нём необходимо освобождать ресурсы, удерживаемые провайдером: закрывать соединения с базой данных, сбрасывать кэши, завершать транзакции.

Таким образом, жизненный цикл провайдера привязан к сессии: фабрика создаёт новый экземпляр через `ProviderFactory.create(session)` в начале каждой сессии, а Keycloak вызывает `close()` при её завершении.

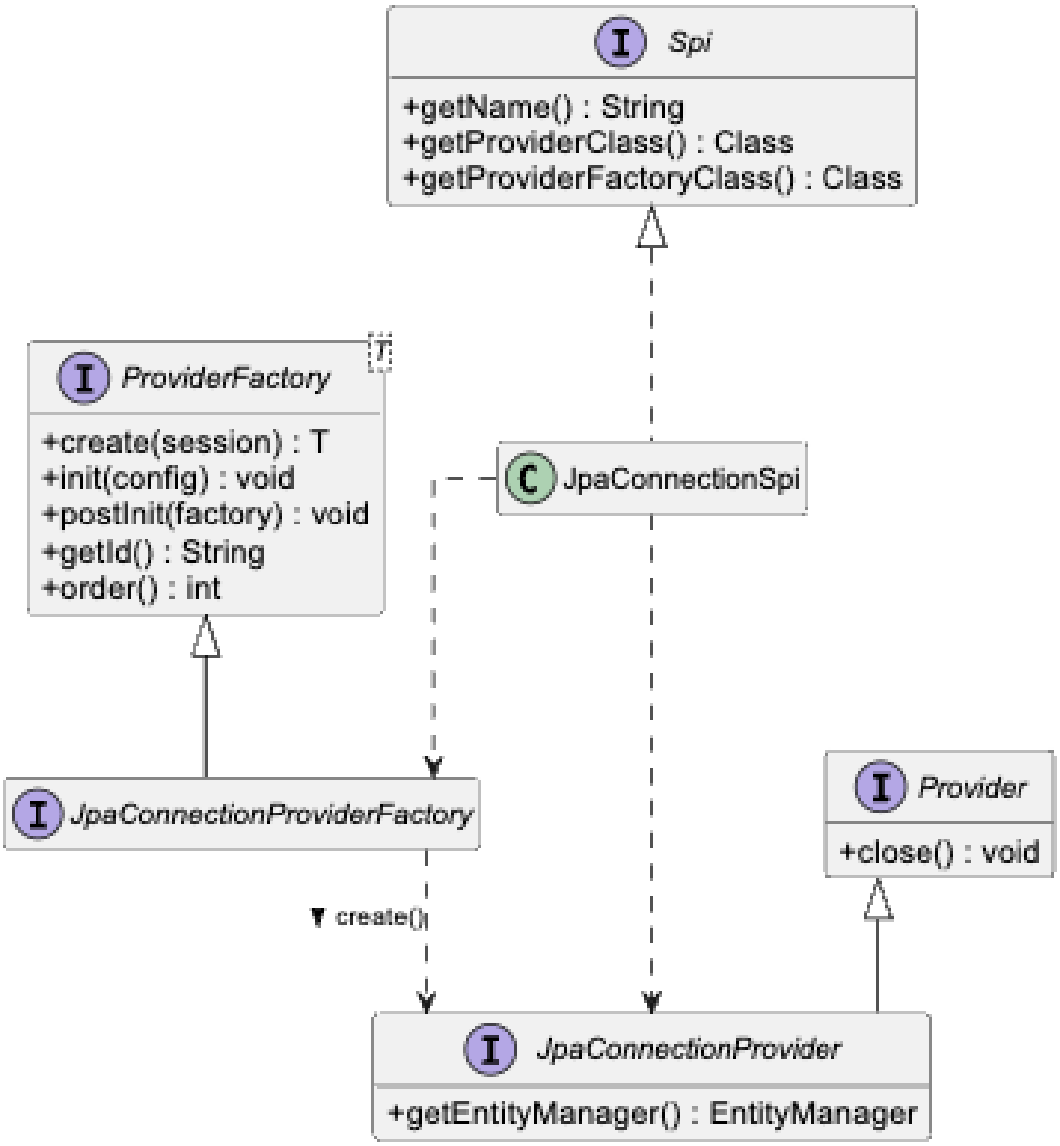

Рисунок 1 – Пример реализации SPI: JPA-подключение в Keycloak

Реальная бизнес-логика реализуется не в самом интерфейсе `Provider`, а в дочерних интерфейсах и их конкретных реализациях.

Например, `JpaRealmProvider` реализует сразу несколько интерфейсов — `RealmProvider`, `ClientProvider`, `GroupProvider`, `RoleProvider` — и реализует всю работу с соответствующими сущностями на уровне базы данных через JPA. Каждый метод такого провайдера выполняет конкретную операцию: создание, поиск, обновление или удаление объектов в рамках текущей сессии и транзакции.

Выбор провайдера по приоритету

Алгоритм выбора следующий:

а) Если явно задан провайдер через конфигурацию, то используется он;
б) Если зарегистрирована только одна фабрика, то используется она;
в) Если зарегистрировано несколько фабрик, то выбирается та, у которой наибольшее значение `order()` среди всех с `order() > 0`.
г) Иначе используется фабрика с идентификатором default.

1.1.3. Ключевые SPI и ProviderFactory для работы с базой данных

— `JpaConnectionProviderFactory` — управляет созданием соединений с базой данных и жизненным циклом JPA `EntityManagerFactory`;
— `LiquibaseConnectionProviderFactory` — управляет подключением Liquibase для выполнения миграций;
— `DBLockProviderFactory` — реализует распределённую блокировку для синхронизации при параллельном старте нескольких инстанций Keycloak;
— `RealmProviderFactory`, `ClientProviderFactory`, `ClientScopeProviderFactory` и тд — фабрики провайдеров для работы с основными сущностями данных.

1.1.4. Механизм миграций и DBLockProvider.

При старте Keycloak фабрика `JpaConnectionProviderFactory` инициирует создание схемы базы данных и выполнение миграций через Liquibase. Поскольку Keycloak может запускаться в виде нескольких параллельных инстанций, необходима синхронизация: только один узел должен выполнять миграции в каждый момент времени. Эту роль выполняет `DBLockProvider` — провайдер глобальной распределённой блокировки, хранящий состояние в отдельной таблице базы данных.

Блокировки разделены по пространству имён (*namespace*):

— `DATABASE` — захватывается перед применением Liquibase-миграций (создание и обновление схемы таблиц);
— `KEYCLOAK_BOOT` — захватывается после завершения DDL-миграций для выполнения миграций данных (модельных миграций Keycloak).

Каждый узел ожидает снятия блокировки (по умолчанию до 900 секунд), после чего продолжает загрузку.

1.1.5. Модель данных Keycloak

Keycloak хранит данные в реляционных таблицах, отображаемых через Hibernate-аннотации на JPA-сущности.

Основные сущности включают:

— **Realm** — область, содержащая конфигурацию, пользователей, клиентов, роли и группы;
— **Client** — OAuth/OIDC-клиент, зарегистрированный в realm;
— **ClientScope** — область видимости, определяющая набор claims в токене;
— **Role** — роль, назначаемая пользователям или клиентам;
— **Group** — группа пользователей с наследуемыми ролями;
— **UserEntity** — пользователь с атрибутами, учётными данными и привязкой к внешним провайдерам идентификации;

Схема Keycloak включает более 80 таблиц и активно использует внешние ключи для обеспечения ссылочной целостности. Это является принципиальным ограничением при переносе на YDB.

1.2. СУБД YDB

YDB — это NewSQL-база данных, разработанная Яндексом и опубликованная как открытое программное обеспечение в 2022 году. YDB сочетает свойства реляционных СУБД (строгая типизация, SQL-запросы, ACID-транзакции) с возможностями распределённых систем (горизонтальное масштабирование, автоматическое шардирование).

1.2.1. Ключевые свойства YDB

Транзакционная модель. YDB поддерживает сериализуемую изоляцию транзакций и использует *оптимистичную блокировку*: конфликты не предотвращаются превентивно, а обнаруживаются в момент коммита. Если за

время выполнения транзакции другая транзакция успела изменить затронутые строки, сервер откатывает её и возвращает ошибку `Transaction lock invalidated`.

YDB классифицирует ошибки на два типа. *Non-retryable*-ошибки указывают на логические нарушения (например, превышение схемных ограничений), и повтор бессмысленен. *Retryable*-ошибки носят временный характер: конфликт устранится сам собой, если приложение повторит транзакцию с нуля. Ошибка `Transaction lock invalidated` относится ко второму типу, однако повторяемая операция должна быть *идемпотентной* — повторный вызов не должен приводить к дублированию эффектов.

Отсутствие поддержки FOR UPDATE. YDB не реализует пессимистичные блокировки строк. Запросы с `SELECT ... FOR UPDATE` или `SELECT ... FOR SHARE` являются синтаксически некорректными в YQL.

Обязательный первичный ключ. Каждая таблица в YDB обязана иметь явно определённый первичный ключ. Таблицы без первичного ключа создать невозможно.

Отсутствие внешних ключей. YDB не поддерживает декларативные ограничения внешних ключей. Ссылочная целостность должна обеспечиваться на уровне приложения.

Диалект YQL. YDB использует YQL (Yandex Query Language) — диалект SQL с рядом ограничений:

— не поддерживаются ссылки на столбцы по порядковому номеру (`ORDER BY 1`);

— не поддерживается `CONCAT()`, вместо них используется оператор `||`;

— не поддерживаются операции `lower` и `upper`, вместо них используются `Unicode::ToLower` и `Unicode::ToUpper`

— предикат `JOIN ON` обязан ссылаться на столбцы обеих сторон соединения; условия фильтрации по одной таблице необходимо выносить в `WHERE`;

— неявные декартовы произведения (перечисление таблиц через запятую в `FROM`) не поддерживаются — вместо них требуется явный `JOIN`.

1.2.2. Диалект Hibernate для YDB

Hibernate — это ORM-фреймворк для Java, который отображает объекты на реляционные таблицы и абстрагирует работу с базой данных от конкретного

SQL-диалекта. Для взаимодействия с конкретной СУБД Hibernate использует *диалект* — класс, производный от `org.hibernate.dialect.Dialect`. Диалект отвечает за отображение Java-типов на SQL-типы конкретной СУБД и генерацию корректных SQL-выражений в её синтаксисе.

Для YDB существует реализация диалекта только для Hibernate 6, однако она содержит ряд неточностей в обработке YQL-специфичных конструкций. Поддержки Hibernate 7, используемого в Keycloak 26, не существовало.

1.3. Аналоги и существующие решения

На сегодняшний день существует реализация альтернативного хранилища для Keycloak на базе Apache Cassandra — проект `keycloak-cassandra-extension`.

Проект реализует замену стандартного JPA-хранилища на Apache Cassandra — распределённую NoSQL СУБД. Интеграция выполнена через механизм SPI Keycloak: расширение предоставляет собственные реализации `DatastoreProvider` и отдельных провайдеров для каждого типа сущностей (`UserProvider`, `RealmProvider`, `ClientProvider` и других). Для работы с базой данных используется напрямую DataStax Java Driver, без Hibernate и JPA.

Ключевым отличием является то, что Cassandra не поддерживает реляционные транзакции. Для обеспечения согласованности при конкурентных обновлениях используются *легковесные транзакции* (Lightweight Transactions, LWT) — операции типа compare-and-set над отдельными строками. Каждая сущность хранит поле `version`, и обновление выполняется только при совпадении версии: если другой процесс успел изменить запись, операция отклоняется, и клиент должен повторить её.

Отсутствие реляционной модели потребовало полного переосмысления схемы данных. Если в стандартном Keycloak каждое поле сущности соответствует отдельной колонке таблицы (например, `JpaRealmEntity` содержит десятки типизированных колонок), то в `keycloak-cassandra-extension` большинство свойств сущностей хранится в обобщённых коллекциях — ассоциативных массивах атрибутов (`map<text, list<text»`). Это позволяет не менять схему при добавлении новых полей, но лишает возможности использовать SQL-поиск по конкретным полям. Для поддержки поиска пользователей создаются отдель-

ные индексные таблицы по выбранным атрибутам; поиск по произвольным атрибутам требует полного обхода базы данных.

Проект поддерживает основные функции Keycloak, однако не реализует Authorization Services и организации (Organizations). Миграции схемы выполняются через библиотеку Cassandra Migration.

Таким образом, `keycloak-cassandra-extension` подтверждает принципиальную возможность замены реляционного хранилища в Keycloak через механизм SPI, однако потребовал полной переработки модели данных. Интеграция с YDB принципиально отличается: YDB является реляционной СУБД с JDBC-драйвером и поддержкой Hibernate, что позволяет сохранить существующую схему данных Keycloak. Основной задачей становится адаптация Hibernate-диалекта к YQL и корректная обработка оптимистичных транзакций YDB.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ РЕШЕНИЯ

2.1. Общая архитектура модуля

Модуль `keycloak-ydb-extension` встраивается в Keycloak через механизм SPI и состоит из трёх подмодулей Maven:

— `core/` — основная реализация расширения на Kotlin, упакованная как shaded JAR и размещаемая в директории `providers/` Keycloak;

— `retry-proxy/` — HTTP-прокси на Ktor, перехватывающий ответы с кодом 503 и повторяющий запрос при временных ошибках YDB;

— `test/` — интеграционные тесты на основе JUnit и `ydb-junit5-support` с контейнером YDB.

Точкой входа служат файлы регистрации провайдеров в `META-INF/services/`, переопределяющие стандартные JPA- и Liquibase-провайдеры Keycloak.

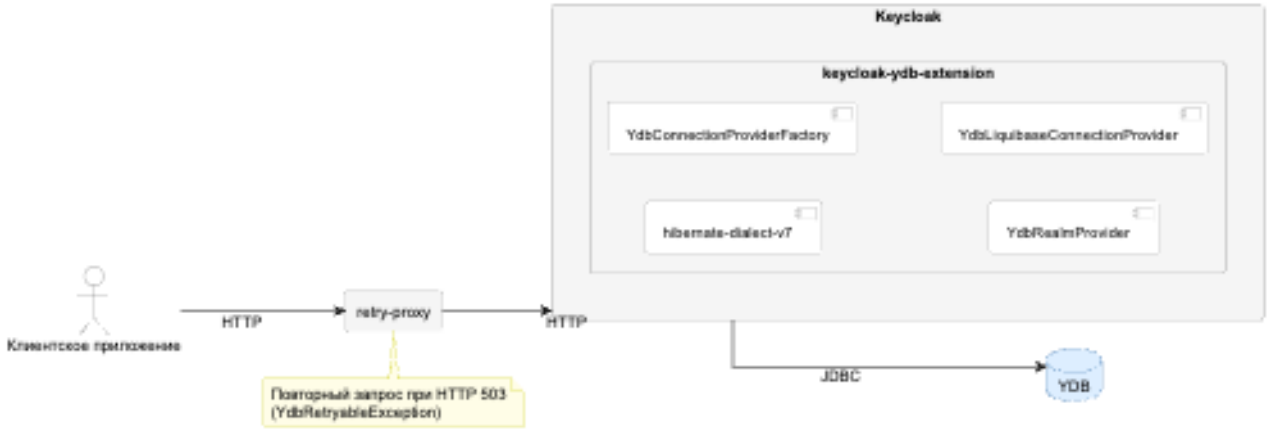

Рисунок 2 – Схема использования модуля keycloak-ydb-extension

2.2. Принципы проектирования схемы данных

2.2.1. Анализ подходов к миграции схемы

На начальном этапе были рассмотрены два подхода к созданию схемы базы данных.

Подход 1: стандартные Liquibase-миграции Keycloak. Keycloak хранит историю изменений схемы в виде наборов миграций Liquibase, накопленных за всю историю проекта (с версии 1.0). Набор включает 77 XML-файлов, часть из которых содержит ветвление по типу СУБД (`dbms="postgresql"`, `dbms="mssql"`, `dbms="oracle"` и т. д.) и условные сценарии миграции с `onFail="MARK_RAN"`. YDB в этом списке не предусмотрен, поэтому при попытке применить стандартные миграции Liquibase выдаёт ошибки валидации.

Миграции рассчитаны исключительно на поддерживаемые СУБД и активно используют конструкции, которые YDB не поддерживает:

— таблицы без явного первичного ключа (например, `SCOPE_MAPPING`, `REDIRECT_URIS`);
— добавление первичного ключа отдельной командой после `CREATE TABLE`;
— декларативные ограничения `FOREIGN KEY`;
— ограничения `UNIQUE` — вместо них YDB поддерживает индексы `GLOBAL UNIQUE`;
— добавление `NOT NULL` на существующую колонку через `ALTER TABLE` — `NOT NULL` поддерживается только при создании таблицы;
— добавление значения по умолчанию на существующую колонку через отдельную операцию `addDefaultValue` — YDB поддерживает `DEFAULT` только при создании таблицы;
— операция `RENAME COLUMN`.

Подход 2: автоматическая генерация схемы через Hibernate (`HBM2DDL_AUTO=update`). Hibernate умеет генерировать DDL напрямую из аннотаций JPA-сущностей. Этот подход позволил создать некоторые таблицы, однако миграция не завершается успешно из-за того, что в таблицах не было первввчиных ключей

Принятое решение: собственные Liquibase-миграции. Оба автоматических подхода оказались неприменимы без существенной доработки. Принято решение написать собственный набор миграций (82 SQL-файла), воспроизводящих схему Keycloak с учётом ограничений YDB. За основу взята финальная схема PostgreSQL, полученная после применения всей истории стандартных Liquibase-миграций Keycloak.

При переносе схемы были применены следующие адаптации:

— **Составные первичные ключи.** Связующие таблицы, не имеющие суррогатного ключа в PostgreSQL-схеме (например, `SCOPE_MAPPING`, `REDIRECT_URIS`, `USER_ROLE_MAPPING`), получили составной первичный ключ из колонок, образующих естественный идентификатор строки.

— **Замена `UNIQUE`-ограничений на `GLOBAL UNIQUE`-индексы.** Все уникальные ограничения PostgreSQL заменены индексами вида `GLOBAL UNIQUE`, обеспечивающими ту же семантику уникальности в YDB.

— **Удаление `FOREIGN KEY`-ограничений.** Объявления внешних ключей удалены; ссылочная целостность обеспечивается на уровне приложения. Для сохранения производительности поисковых запросов на колонках, бывших внешними ключами, созданы обычные `GLOBAL`-индексы.

2.3. Разработка диалекта Hibernate для YDB

На начальном этапе предполагалось использовать существующий `hibernate-dialect-v6`, поскольку в зависимостях Keycloak явно упоминался Hibernate 6. Анализ зависимостей показал, что Keycloak 26 построен на Quarkus, который транзитивно подтягивает Hibernate 7. Исправления, внесённые в `hibernate-dialect-v6`, затрагивали API, несовместимый с Hibernate 7, и не применялись. Таким образом, для корректной работы с Keycloak 26 потребовалось создать отдельный модуль `hibernate-dialect-v7`.

Для обхода ограничения `FOR UPDATE` рассматривались два варианта: переопределение конкретных методов провайдеров и добавление в диалект настройки глобального игнорирования lock hints. Второй вариант принят как менее трудоёмкий и не требующий изменений в коде провайдеров Keycloak.

2.4. Архитектура механизма повторного выполнения транзакций

ГЛАВА 3. РЕАЛИЗАЦИЯ

3.1. Диалект Hibernate v7 для YDB

Модуль `hibernate-dialect-v7` реализует класс `YdbDialect`, расширяющий базовый диалект Hibernate 7 для работы с YDB. В ходе интеграции с Keycloak были выявлены и устранены следующие несовместимости.

Обработка `OFFSET` без `LIMIT`. Документация YDB описывает `OFFSET` не указывает явно на обязательность `LIMIT`. На практике же запрос с `OFFSET` без `LIMIT` завершается ошибкой. Keycloak в ряде запросов указывает только `OFFSET`, что приводило к сбою при выполнении. Поэтому В `YdbSqlAstTranslator` добавлена логика подстановки `LIMIT Long.MAX_VALUE` при наличии `OFFSET` без `LIMIT`.

Порядковые ссылки в `ORDER BY`. YDB не поддерживает конструкции рода `SELECT * FROM TABLE ORDER BY 1` (ссылка на столбец по порядковому номеру). Поэтому в диалекте переопределён метод `supportsOrdinalSelectItemReference()`, что вынуждает Hibernate генерировать явные имена столбцов.

Строковые функции. YDB не поддерживает стандартные функции `lower()`, `upper()` и `concat()`. В диалекте они переопределены: `lower`/`upper` транслируются в `Unicode::ToLower`/`Unicode::ToUpper`, `concat` — в оператор `||`.

Игнорирование lock hints. Добавлена настройка `hibernate.dialect.ydb.ignore_lock_hints`. При значении `true` методы генерации SQL возвращают запрос без `FOR UPDATE`, что позволяет использовать Keycloak без переопределения провайдеров, вызывающих `PESSIMISTIC_WRITE`-блокировки.

Все изменения приняты в репозиторий `ydb-platform/ydb-java-dialects`.

3.2. Реализация провайдеров Keycloak

YdbConnectionProviderFactoryImpl

3.3. Управление схемой данных через Liquibase

YdbLiquibaseConnectionProvider

3.4. Переопределение именованных запросов

3.5. Тестирование

ЗАКЛЮЧЕНИЕ

TODO